UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed Systems

Remote invocation

David.Villa@uclm.es / Felix.Villanueva@uclm.es / Soledad.Escolar@uclm.es

Contents

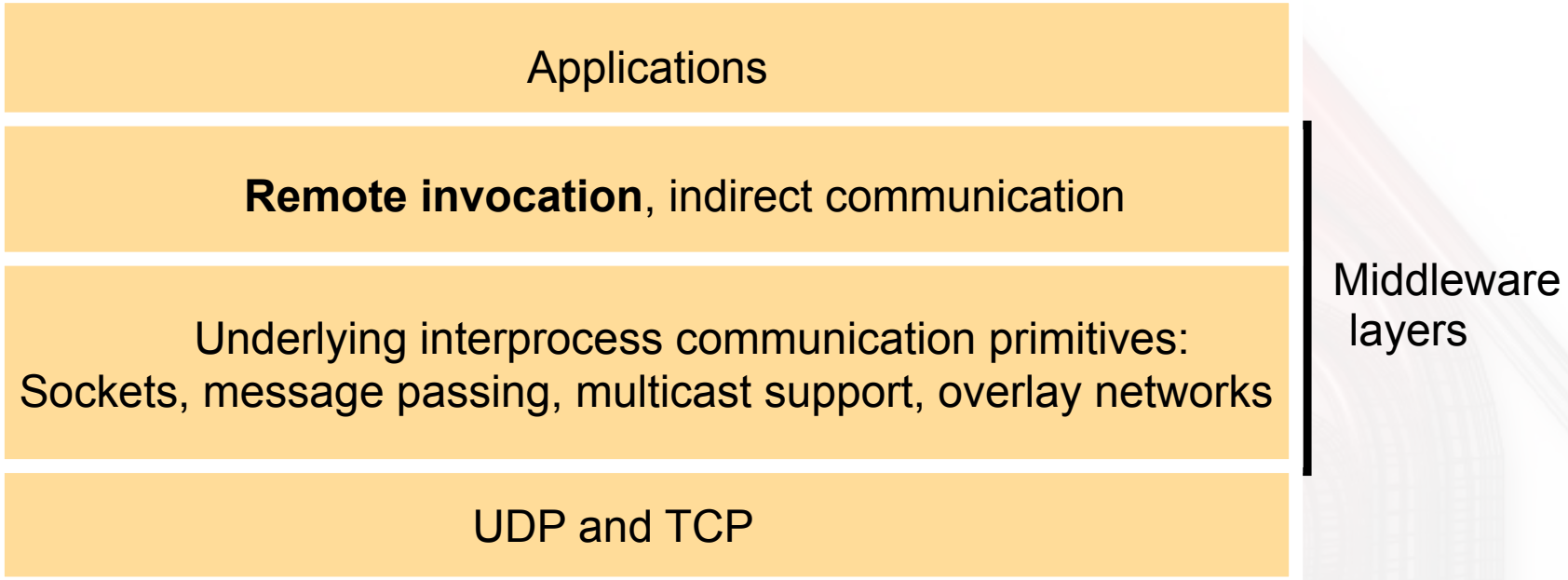
- Request-reply protocols
- Failure tolerance measures
- Idempotent operations
- Invocation semantics
- RPC: pico-rpc, ONC RPC, gRPC
- RMI: pico-rmi, Java RMI, ZeroC Ice
- Service location
- Asynchronous invocation and dispatching
- Performance
- Cap'n Proto
- REST

Tip

All listings in this topic are available for download at:

`https://github.com/UCLM-ESI/ssdd.examples`

Remote Invocation



© Pearson Education 2012

Remote Invocation

Programming paradigms that allows to call *code blocks* belonging to a (remote) process.

RPC (Remote Procedure Call)

Service provides a set of **procedures** (similar to a library) that can be called remotely.

RMI (Remote Method Invocation)

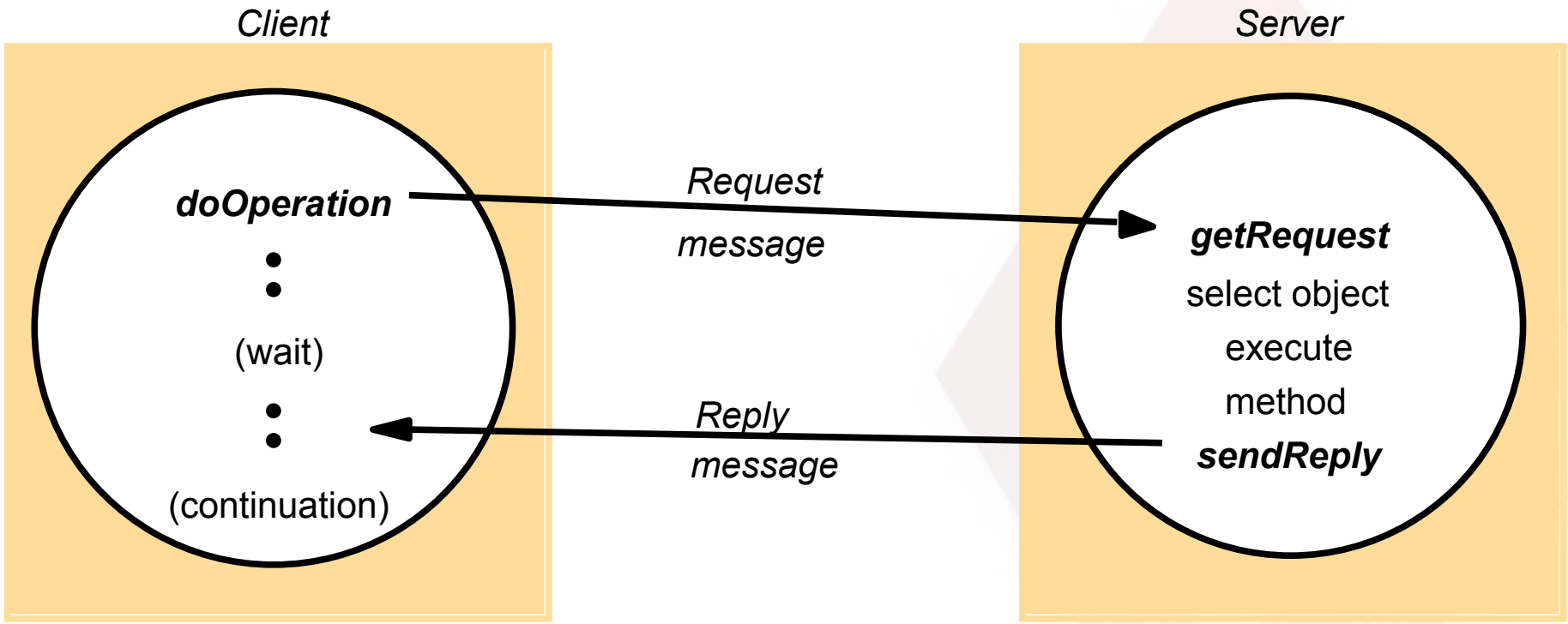
Service provides a set of **objects** (implementing class interfaces) that can be called remotely.

RPC/RMI are supported by:

- ***Request-reply protocols***, that are supported by:
  - ***Message passing* primitives** (send/receive)

Request-reply protocols

Two way exchange of messages (client-server model)



© Pearson Education 2012

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Request-reply protocols
Design considerations

- blocking vs. non-blocking
- reliable vs. unreliable
- features:
  - Message identifiers
  - Timeouts
  - Duplicate requests
  - Lost reply messages
  - History (store replies to avoid re-execution)

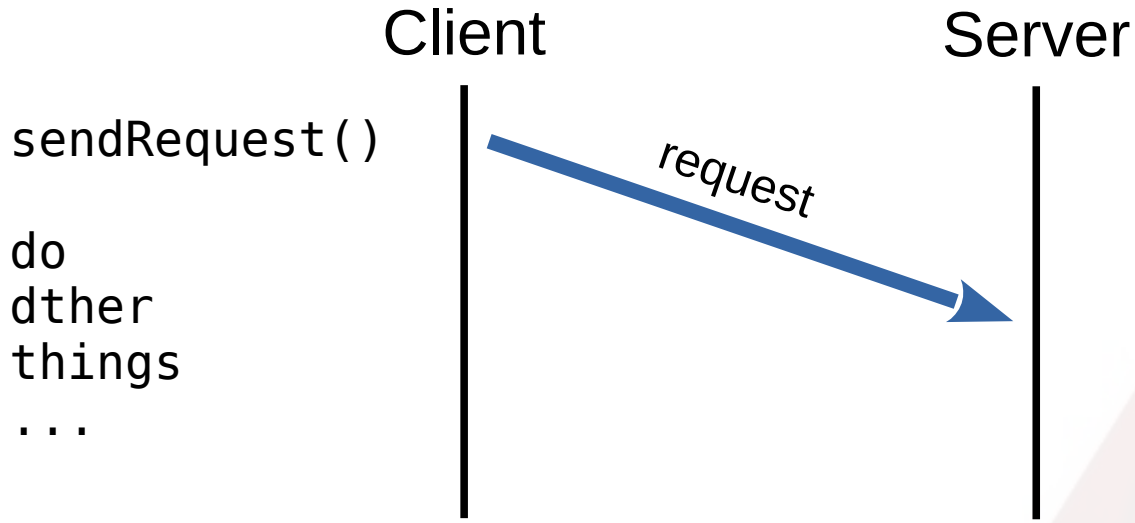
Styles of exchange protocols

Three protocol styles that provide different types request behavior when communication errors arise:

- **Request (R)**
- **Request-reply (RR)**
- **Request-reply-ack (RRA)**

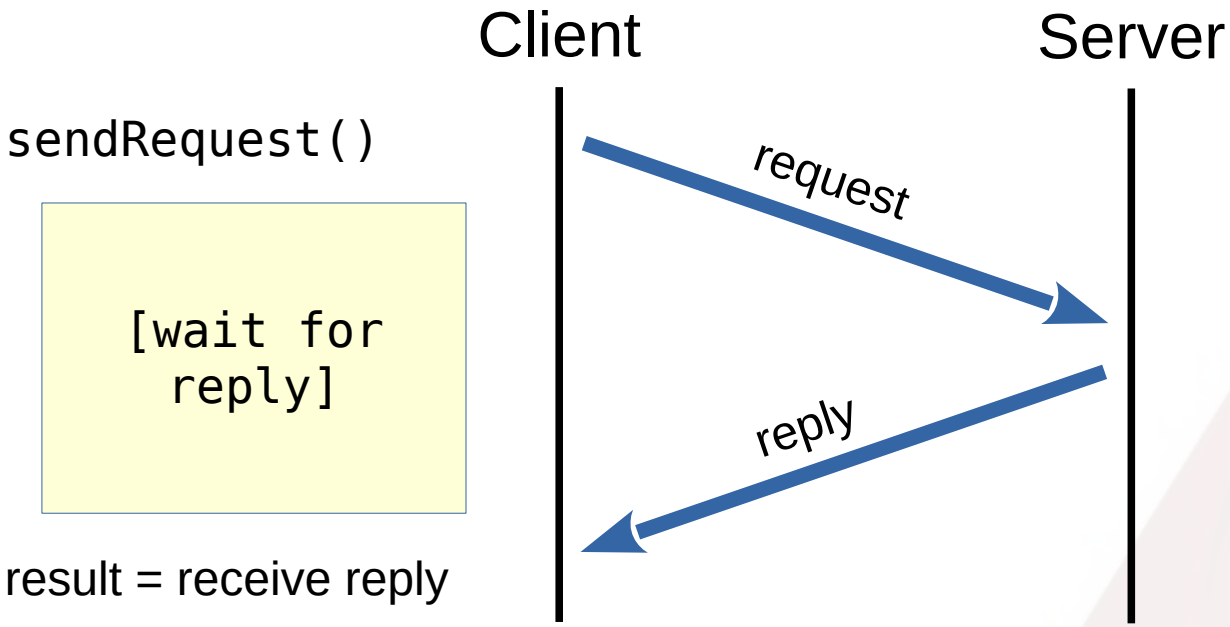
	Messages sent by		
	Client	Server	Client
R	Request		
RR	Request	Reply	
RRA	Request	Reply	Acknowledge reply

Request (R)



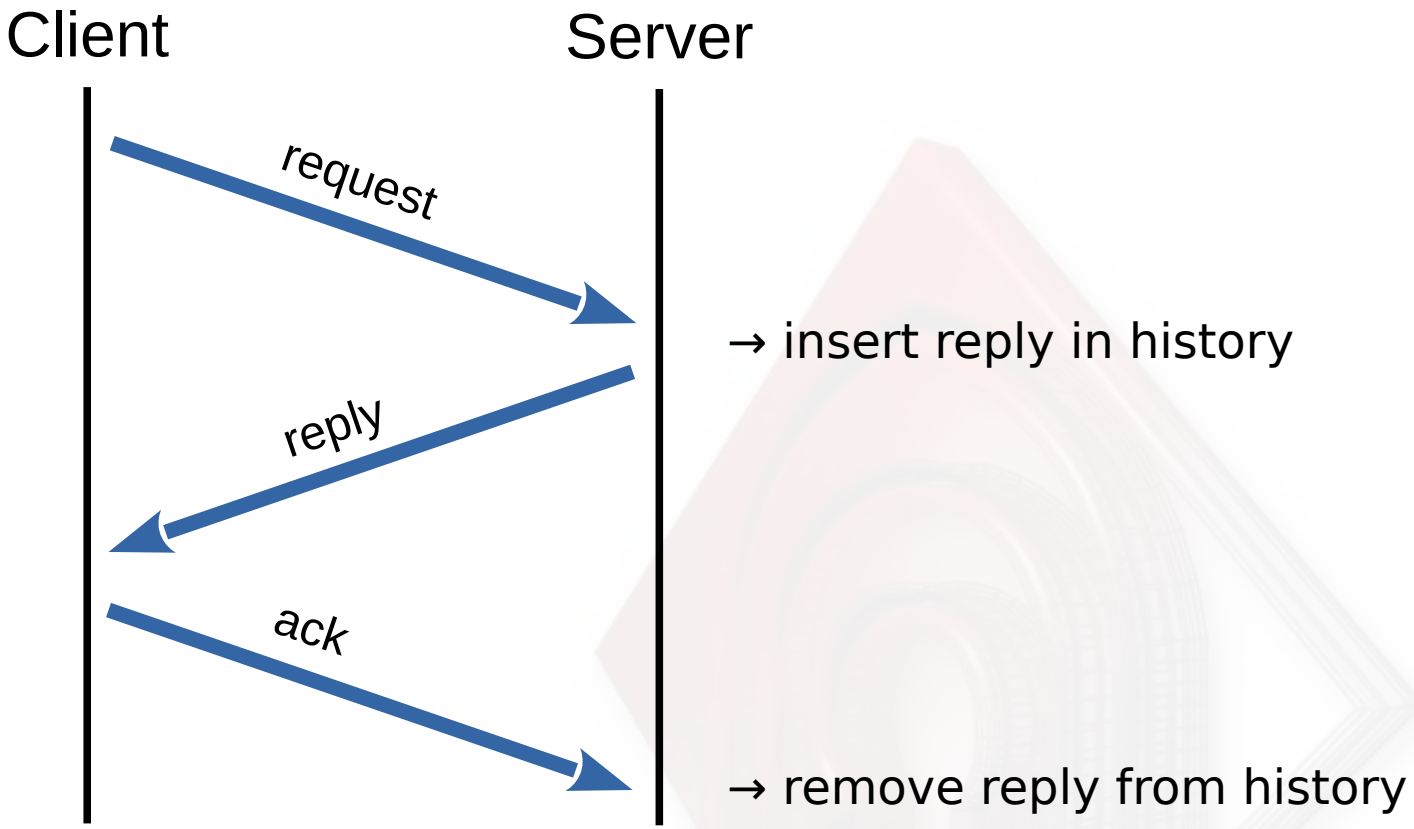
- Server returns nothing
- Client **does not wait** nor require confirmation (non blocking request)
- No timeout nor retransmission
- Typically over UDP

Request-Reply (RR)



- Typically a blocking request (client is blocked waiting for reply)
- Server reply is like an ACK for client
- Next client request is like an ACK for server
- May include timeouts, retransmissions and duplicates control

Request-Reply-Ack (RRA)



- Client send ACK for server reply (including the request-id).
- Support *retransmission of results* at server side.

Failure tolerance measures

Depending on exchange protocols (R/RR/RRA), three fault tolerance measures (delivery guaranties) are available:

1. Request retry

- Client retransmit request message until reply/ACK is received
- Requires timeouts + retransmissions

2. Duplicated filtering

- Server discards duplicated (retried) request messages.
- Requires previous + requestId

3. Result retransmission

- Server retransmit reply message without re-executing
- Requires previous + history

Idempotent operations

An operation is **idempotent** when executing it several times, with the same arguments, has the same effect on the server state as executing it just once.

- It is a property of the semantics of the operation, not of the middleware.
- Idempotent operations need no duplicate filtering, which simplifies the implementation.
- RRA has to cache the reply precisely for non-idempotent operations, which are assumed to be the common case.

Idempotent

```
set_size(25)    stop()    car.get_color()    get_time()
```

Not idempotent

```
push("item")    counter.inc()

send_mail("user@example.com", "Hello")
```

get_time() is idempotent: the returned value changes on each call, but the server state does not.

RPC Call Semantics

maybe: not ACK, not dup filter

- Remote procedure *may* run 1 or 0
- Client *may* get reply or not

at-least-once: ACK, no dup filter

- Remote procedure run 1+
- Client may get 1+ replies

at-most-once: ACK + dup filter + RT reply

- Remote procedure run 1 or 0
- Client get a reply or an *error*

exactly-once

- Possible, but expensive

Fault tolerance measures			Call semantics
request-retry	dup-filter	re-execute/RT reply	
No	Not applicable	Not applicable	**maybe**
Yes	No	Re-execute procedure	**at-least-once**
Yes	Yes	Retransmit reply	**at-most-once**

Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

RPC Call Semantics

DIY

Semantics

maybe

at-most-once

at-least-once

exactly-once

Real examples

DNS: query retransmitted until answered

syslog over UDP: send a log line

Bank transfer: applied once, even after a crash

Print a document: never print it twice

Remote Procedure Call (RPC)

Goal: make distributed programming looks *as close as possible* to conventional programming.

Not necessarily good

That is: **TRANSPARENCY** (Which types?)

- Calling a procedure that exists in a **remote process memory space** should look similar to local call.
- RPC runtime hides most of communication details:
  - Connection/disconnection
  - Marshalling/encoding (depends on arch: endianness)
  - Message passing
  - Parameters and results

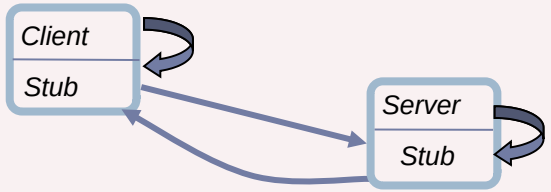
RPC

- Takes cares of:
  - Message specification and parsing
- Not about:
  - Message exchange schema
  - Transport protocol limitations
  - Client/server binding
    - ONC RPC provides that via port-mapper

Important

We name "**RPC**" to the generic remote invocation mechanism,
and "**ONC RPC**" to the specific implementation by Sun Microsystems

Stubs



Client stub

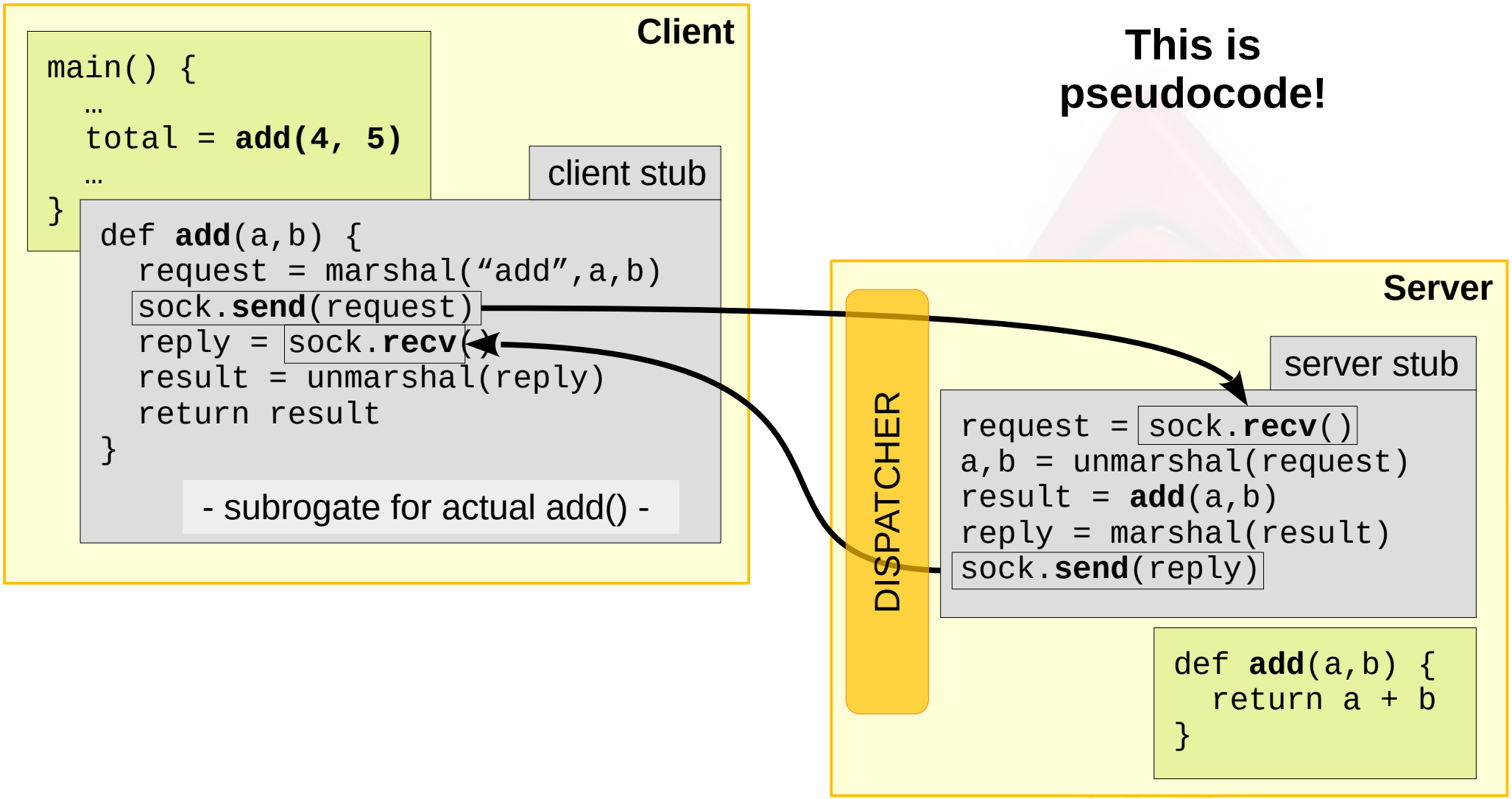
1. The client calls the stub, which locates the server
2. Marshalls the operation and args
3. Sends the request message

9. Receives the reply message
10. Unmarshalls the result and returns it to the caller

Server stub

4. Receives the request; its operation id names the procedure
5. Unmarshalls the input arguments
6. Calls the actual procedure
7. Marshalls the result value
8. Sends the reply message to the client

RPC operation



pico-RPC

DIY

Description: A minimal RPC working example

- **[ssdd.examples:pico-rpc]**

See and play with:
- **client.py**
- **server.py**

Run server:

```
pico-rpc$ ./server.py
Server ready: ('0.0.0.0', 2000)
```

Run client:

```
pico-rpc$ ./client.py localhost 10
factorial(10) = '3628800'
power(10, 9) = '1000000000'
```

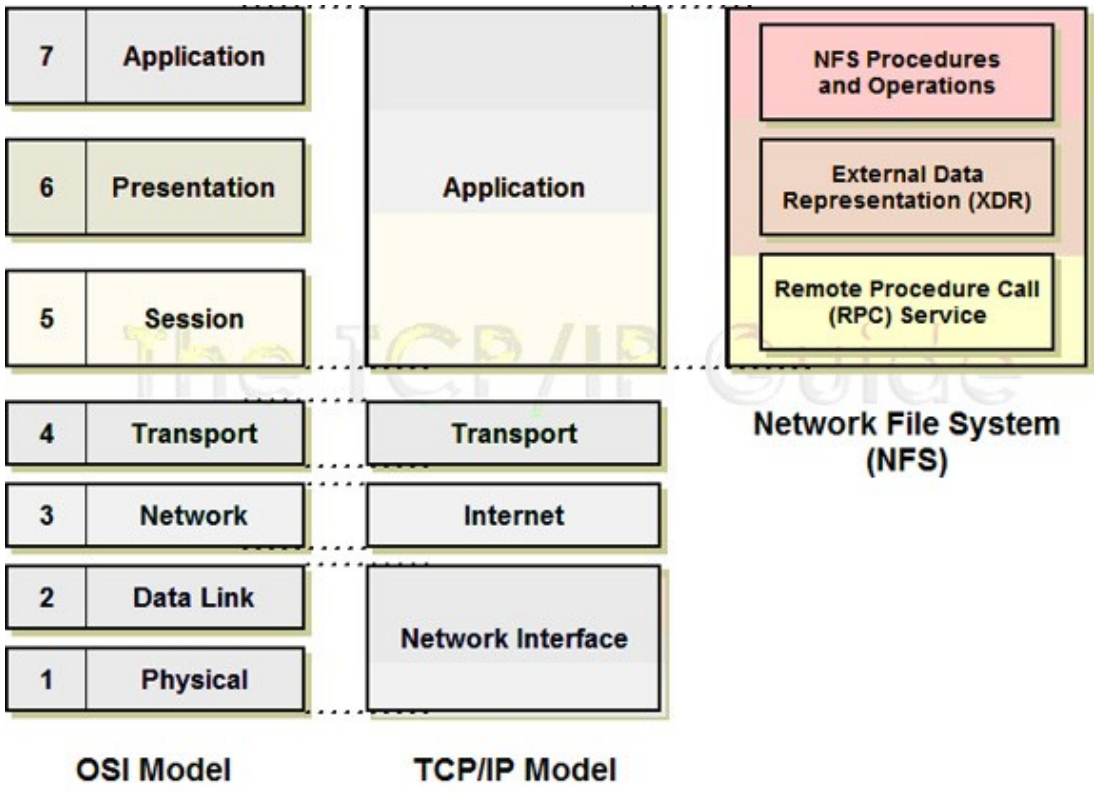
UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Case study
ONC RPC

RFC 1057

- RPC v.2 was standardized by Sun Microsystems
  - Also called “Sun RPC”
- Supports UDP and TCP transports
- Offers *at-least-once* semantics
- A **service** is a set of remote *programs*
- A remote **program** is a set of remote procedures.
- A server may support more than one program version.

ONC RPC in the protocol stack



RPC:

- at the session OSI layer
- In the application TCP/IP layer

Transport protocol independent

- TCP
- UDP

Reliability

- Depends on transport protocol

XDR
eXternal Data Representation

RFC 1832

XDR is the specification language for ONC RPC

- Constants
- Data types including enumerations, structs and unions
- Procedures
  - names, arguments (in/out/inout) and return values
- Programs, versions and procedures:
  - Identified by integers

- The **`rpcgen`** compiler takes a XDR file to generate **stubs** to use on server and client.

XDR :: “Hello World” example

hello.x

```
program MESSAGE_PROG {
  version PRINTMESSAGE_VERS {
    int PRINTMESSAGE(string) = 1;
    int PRINT_MESSAGE_COLOR(string, string) = 2;
  } = 2;

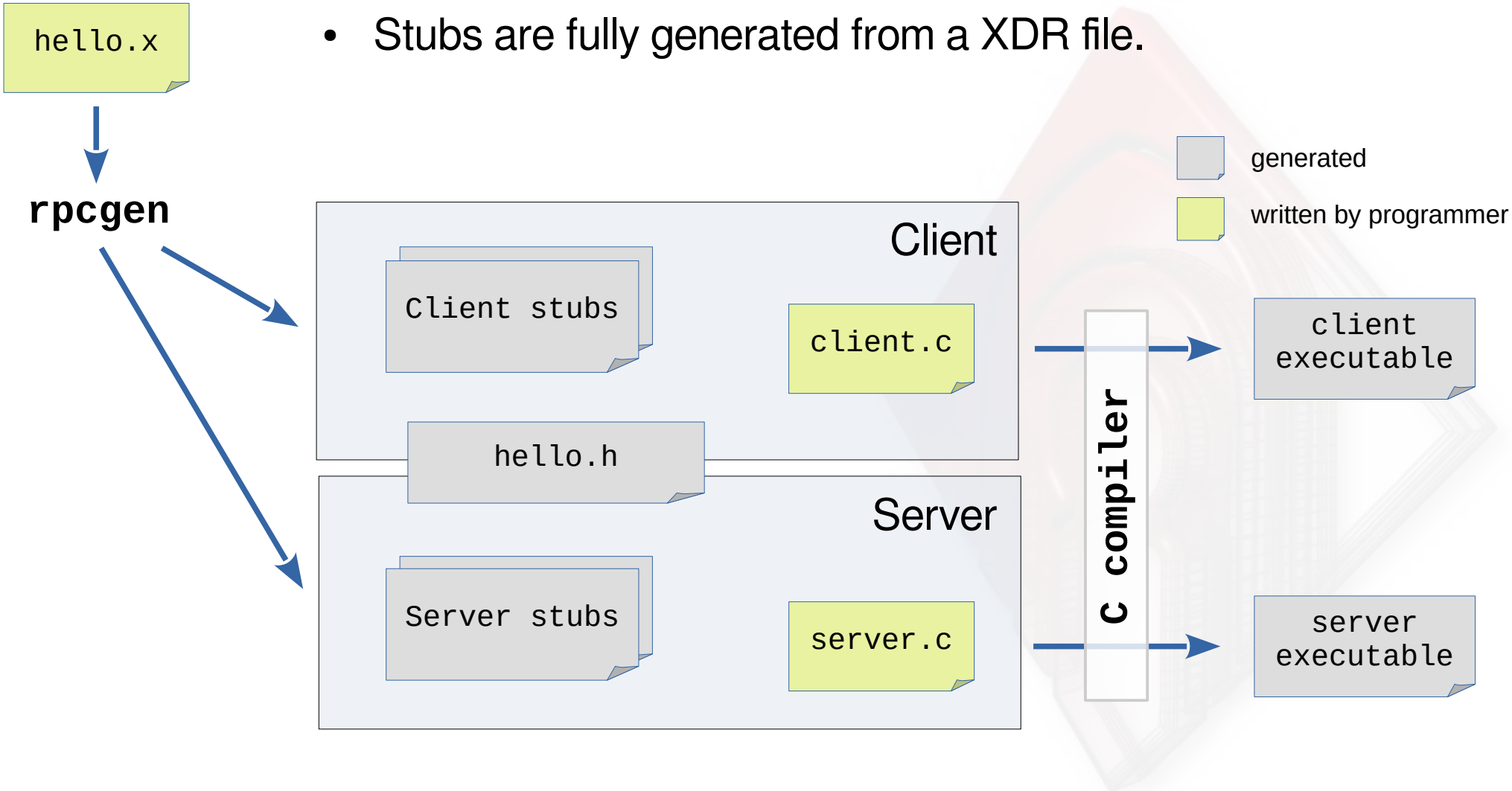

  version PRINTMESSAGE_VERS {
    int PRINTMESSAGE(string) = 1;   /* procedure number */
  } = 1;                            /* version number */
} = 600000000;                      /* program number */
```

The RPC request message is identified by three integers:

- NUM-PROG: remote program number (defined in RFC 1831)
- NUM-VERSION: version number
- NUM-PROCEDURE: procedure number

ONC RPC
Programming model

- Stubs contain the un/marshalling code for a **concrete specification**.
- Stubs are fully generated from a XDR file.



ONC RPC
Example

DIY

Description: A minimal ONC RPC working example

- **[ssdd.examples:onc-rpc-hello/remote]**

See and play with:
- **Makefile**
- **msg.x**
- **client.c**
- **server.c**

Build:

```
onc-rpc-hello$ make
```

Run server:

```
onc-rpc-hello$ ./msg_server
Client sent: 'hello world'
```

Check example:
- **onc-rpc-factorial**

Run client:

```
onc-rpc-hello$ ./msg_client localhost “hello world”
```

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Case study
gRPC

- Protocol Buffers: IDL and binary wire format in one
  - Numbered fields: adding fields does not break older peers
  - **`protoc`** generates stubs for many languages
- One request message in, one reply message out
- Transport is **`HTTP/2`** over TCP, optionally with TLS
  - Each call is a POST to **`/hello.Hello/write`**
  - Concurrent calls multiplexed on a single TCP connection
- No binder: nothing like **`rpcbind`**, the endpoint must be known
- Offers *at-most-once* semantics: no automatic retry by default
- Four call modes: unary, server, client and bidirectional streaming
  - Declared with **`stream`** in the interface

gRPC Example

DIY

Description: A gRPC hello wold example

- **[ssdd.examples:grpc-hello]**

See and play with:
- **server.py**
- **client.py**

Interface:

hello.proto

```
syntax = "proto3";
package hello;
service Hello {
   rpc write (PrintRequest) returns (PrintReply) {}
}
message PrintRequest {
  string message = 1;
}
message PrintReply {}
```

Check example:
- **grpc-factorial**

Run server:

```
grpc-hello$ ./server.py
Client sent: 'hello'
```

Run client:

```
grpc-hello$ ./client.py localhost
```

RMI: Remote Method Invocation

RMI is the object oriented counterpart of RPC.

Includes some OO paradigm features:

- Interface inheritance, not implementation
- Objects implements interfaces → objects work *as servers*
- *Object references* as arguments and return values
- Many OO patterns are applicable: factory, container. etc.
- OO devel methodologies and tools
- Remote exceptions

RMI is the base of Object Oriented Middlewares

Interfaces

- Key feature on RMI middlewares
- Specified in an IDL (Interface Definition Language)
  - Examples: IDL (CORBA), Slice (ZeroC Ice)

An interface for a chat app (Slice):

```
module Chat
{
    interface ChatRoomCallback
    {
        void init(Ice::StringSeq users);
        void send(long timestamp, string name, string message);
        void join(long timestamp, string name);
        void leave(long timestamp, string name);
    }
}
```

Source:http://www.zeroc.com/chat/slice.html

Remote object references

Include information to contact a remote object:

- Endpoint: **host, port, protocol** and optionally other connection details: timeout, encoding, etc.
- Object identifier (universally unique).

Usually, middleware often offers a *stringfied* version:

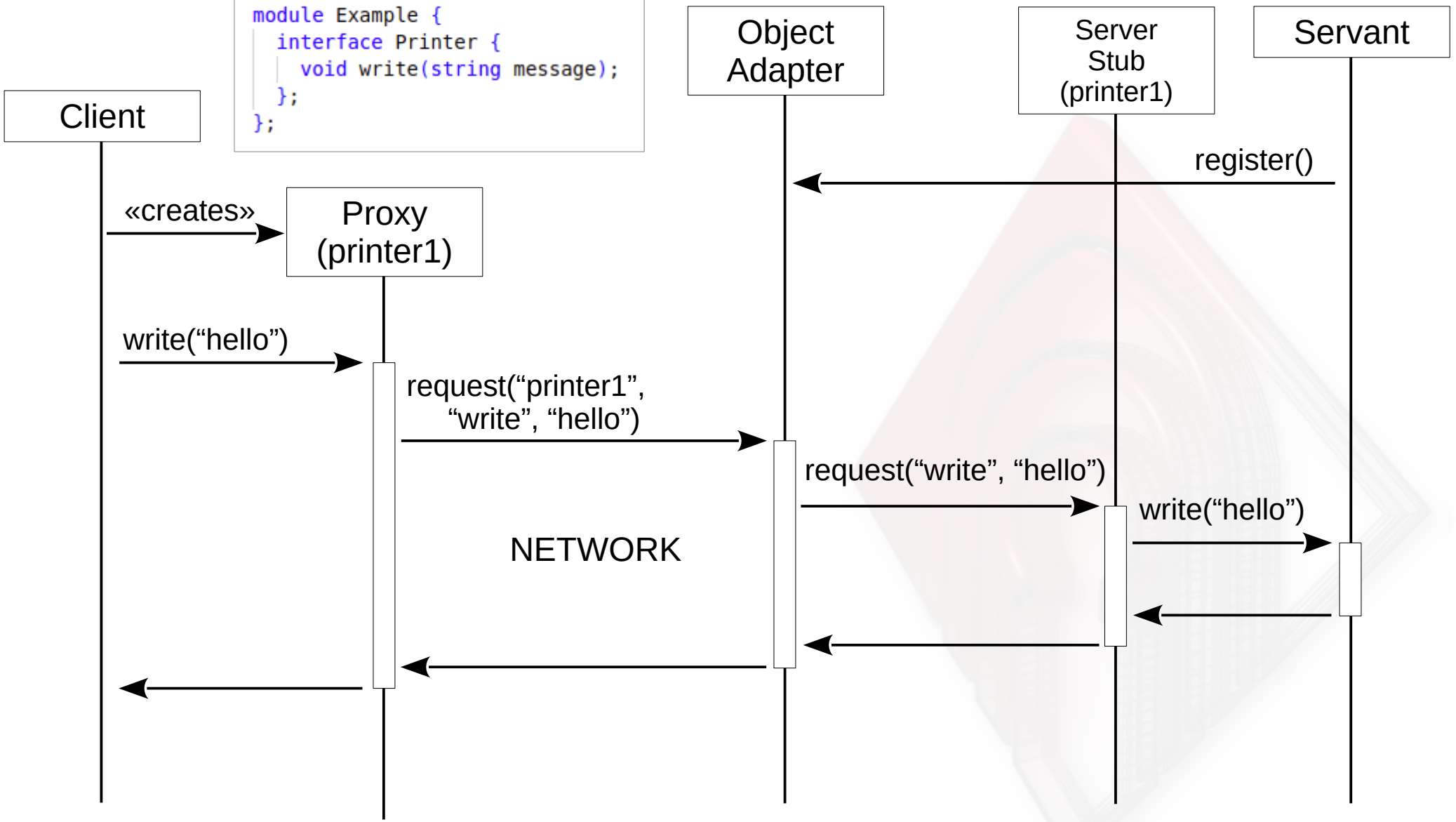
- CORBA: IOR:00585858000000154944C3A69646C746573742F74657...
- CorbaLoc: “corbaloc::160.45.110.41:38693/StandardNS/NameServer-POA/_root”
- ZeroC Ice: “printer1 -t -e 1.1:tcp -h 192.168.8.120 -p 7070 -t 60000”

RMI nomenclature

- **Proxy** is a local subrogate for the remote object. Includes client stubs.
- **Client stubs**: Performs param marshalling and return value unmarshalling.
- **Servant** have the actual method implementations.
- **Object adapter** is the dispatcher, locates corresponding servant and invokes it.
- **Server stubs** performs params unmarshalling and return value (or error) marshalling.
  - Sometimes called “skeleton”, because usually includes a skeleton for the servant implementation.

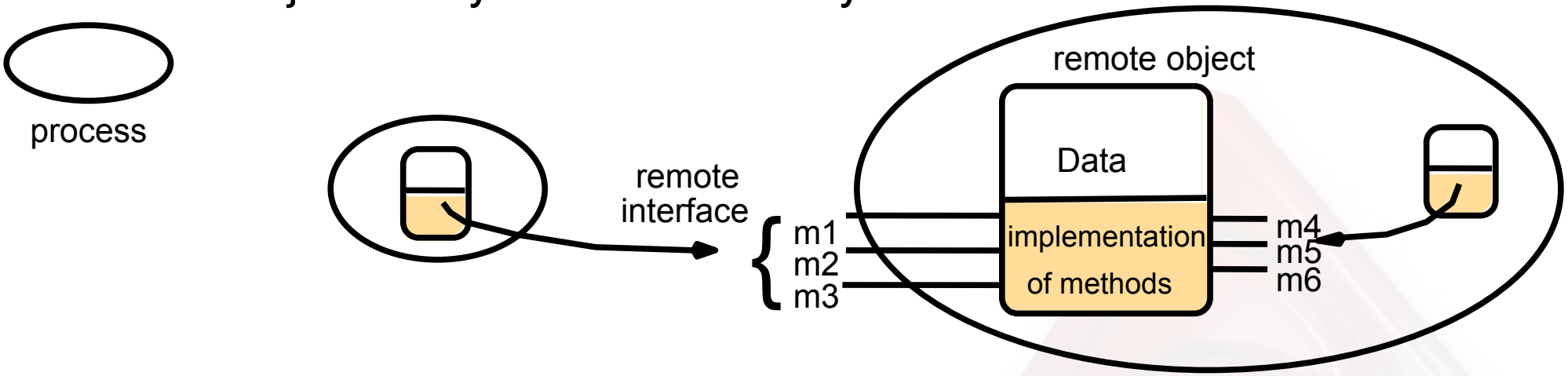
RMI nomenclature in action

```
module Example {
  interface Printer {
    void write(string message);
  };
};
```

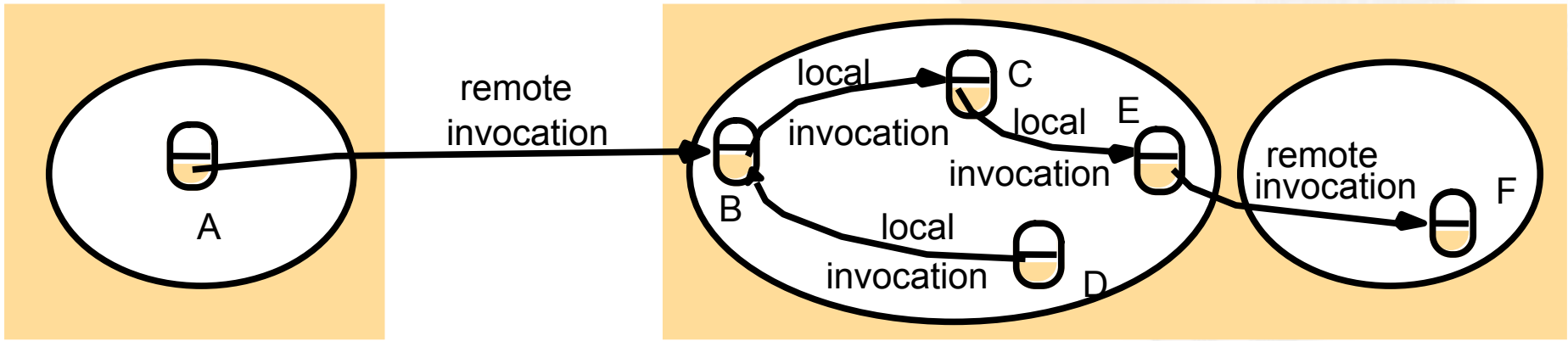


Some remarks

Remote objects may be called "locally":



Invocations among processes are also “remote”



© Pearson Education 2012

pico-RMI

DIY

Description: A minimal RMI working example

- **`[ssdd.examples:pico-rmi]`**

See and play with:
- **`picormi.py`**
- **`rmigen.py`**
- **`server.py`**
- **`client.py`**

Run server:

```
pico-rmi$ ./server.py
Object adapter ready: ('0.0.0.0', 2001)
```

Run client:

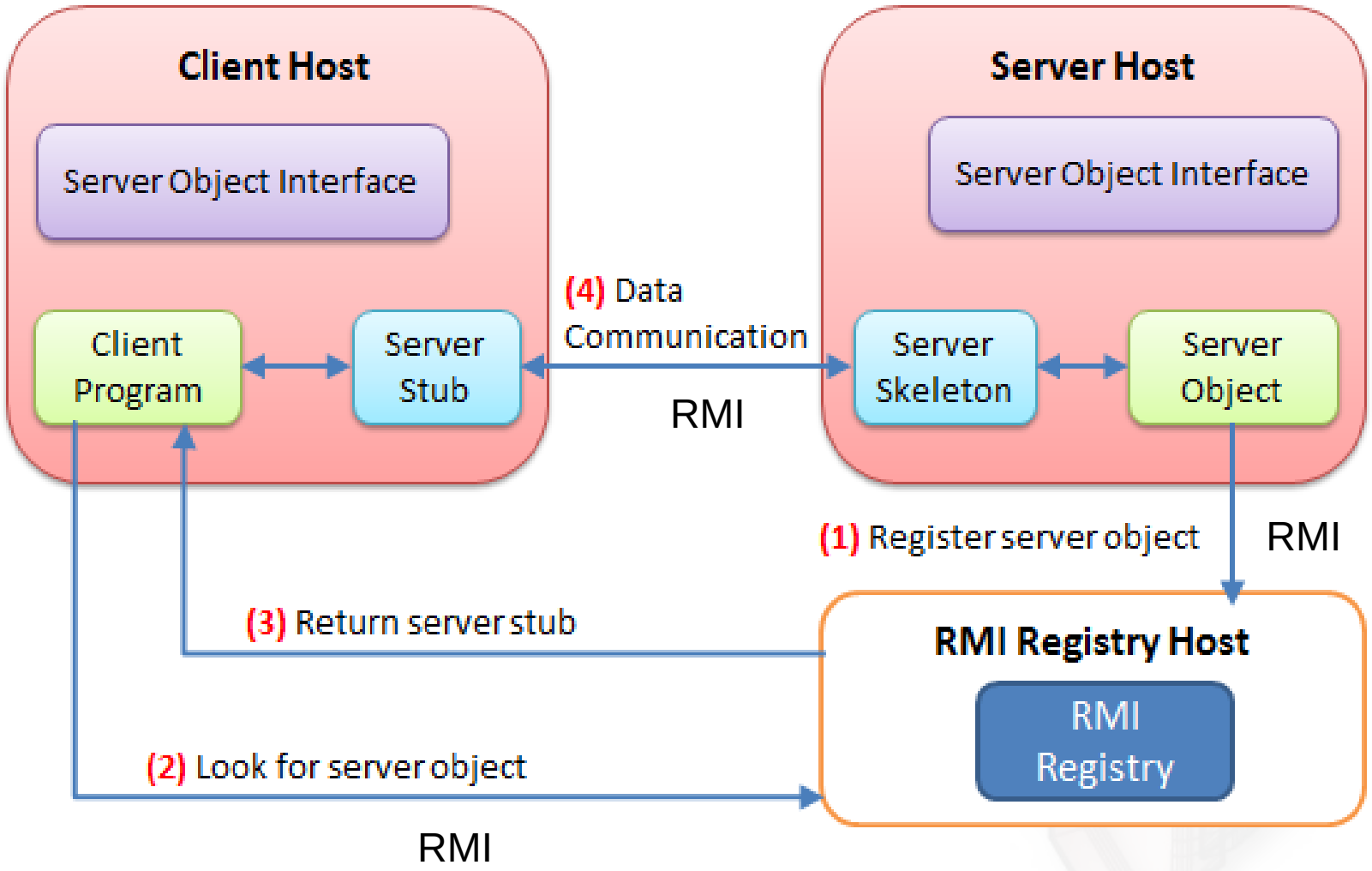
```
pico-rmi$ ./client.py localhost counter1
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Case study
Java RMI

- Interfaces are specified in Java
  - Does not have an IDL. Only support Java language.
  - Based on packages: java.rmi and java.rmi.server
- It offers:
  - Support to build server and objects with remote methods.
  - Mechanism to locate servers.
  - **`rmiregistry`**: is a binder for Java RMI servers.

Java RMI


Client Host
Server Object Interface
Client Program
Server Stub
(4) Data Communication
RMI
Server Host
Server Object Interface
Server Skeleton
Server Object
(1) Register server object
RMI
(3) Return server stub
RMI Registry Host
RMI Registry
(2) Look for server object
RMI

Java RMI
Example

DIY

Description: A Java RMI hello wold

- **[ssdd.examples:java-rmi]**

See and play with:
- **Server.java**
- **Client.java**
- **Makefile**

Interface:

Hello.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote {
  String sayHello() throws RemoteException;
}
```

If you have routing issues, disable network interfaces

Registry:

```
java-rmi$ rmiregistry &
```

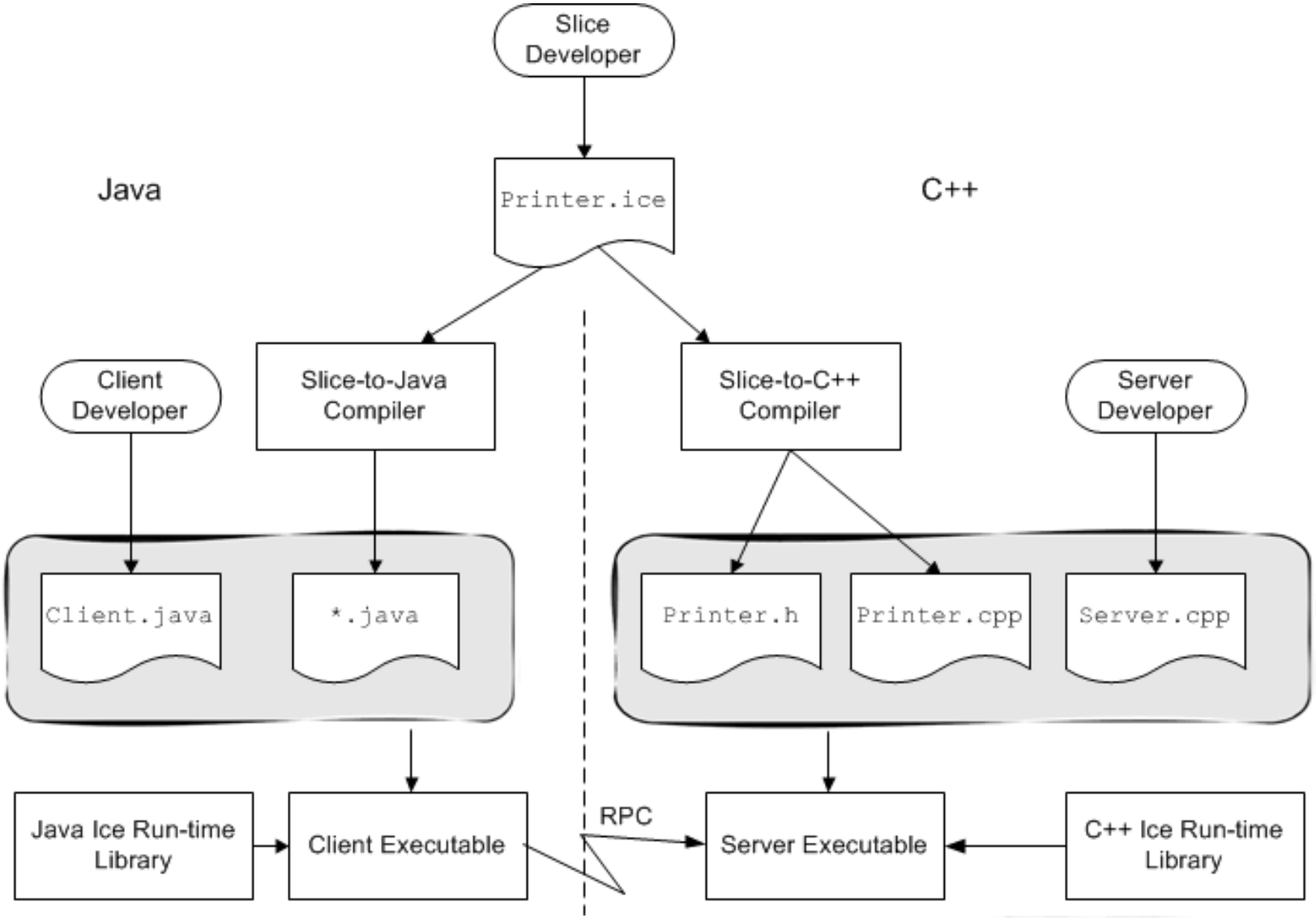
Run server:

```
java-rmi$ make run-server
Server ready
```

Run client:

```
java-rmi$ make run-client
response: Hello, world!
```

Case study
Zeroc Ice



http://doc.zeroc.com/display/Ice/Slice+Compilation

ZeroC Ice
Example

DIY

Description: ZeroC Ice “hello word” with Python

- **[ssdd.examples:ice]**

Interface: **Printer.ice**

```
module Example {
  interface Printer {
    void write(string message);
  };
};
```

See and play with:
- **Printer.ice**
- **Server.py**
- **Server.config**
- **Client.py**

Run server:

```
ice$ ./Server.py -Ice.Config=Server.config
printer1 -t -e 1.1:tcp -h 192.168.8.120 -p 7070
0: Hello World!
```

Run client:

```
ice$ ./Client.py 'printer1 -t -e 1.1:tcp -h 192.168.8.120 -p 7070'
```

Location transparency

- A remote reference includes the server endpoint: it ties the client to a host and a port
- **Location transparency**: the client names the service without saying where it is
  - Servers migrate, replicate or restart elsewhere
- The **binder** maps a stable name to a volatile endpoint
- Bootstrapping: the binder's own endpoint is fixed by convention or configuration
  - `rpcbind`: port 111; Java RMI Registry: port 1099; Ice: `Ice.Default.Locator`
- **Discovery** instead: broadcast the question and let whoever can answer
  - `rpcbind` broadcast call; Ice `LocatorFinder` by multicast
  - No binder address to configure, but limited to the broadcast scope

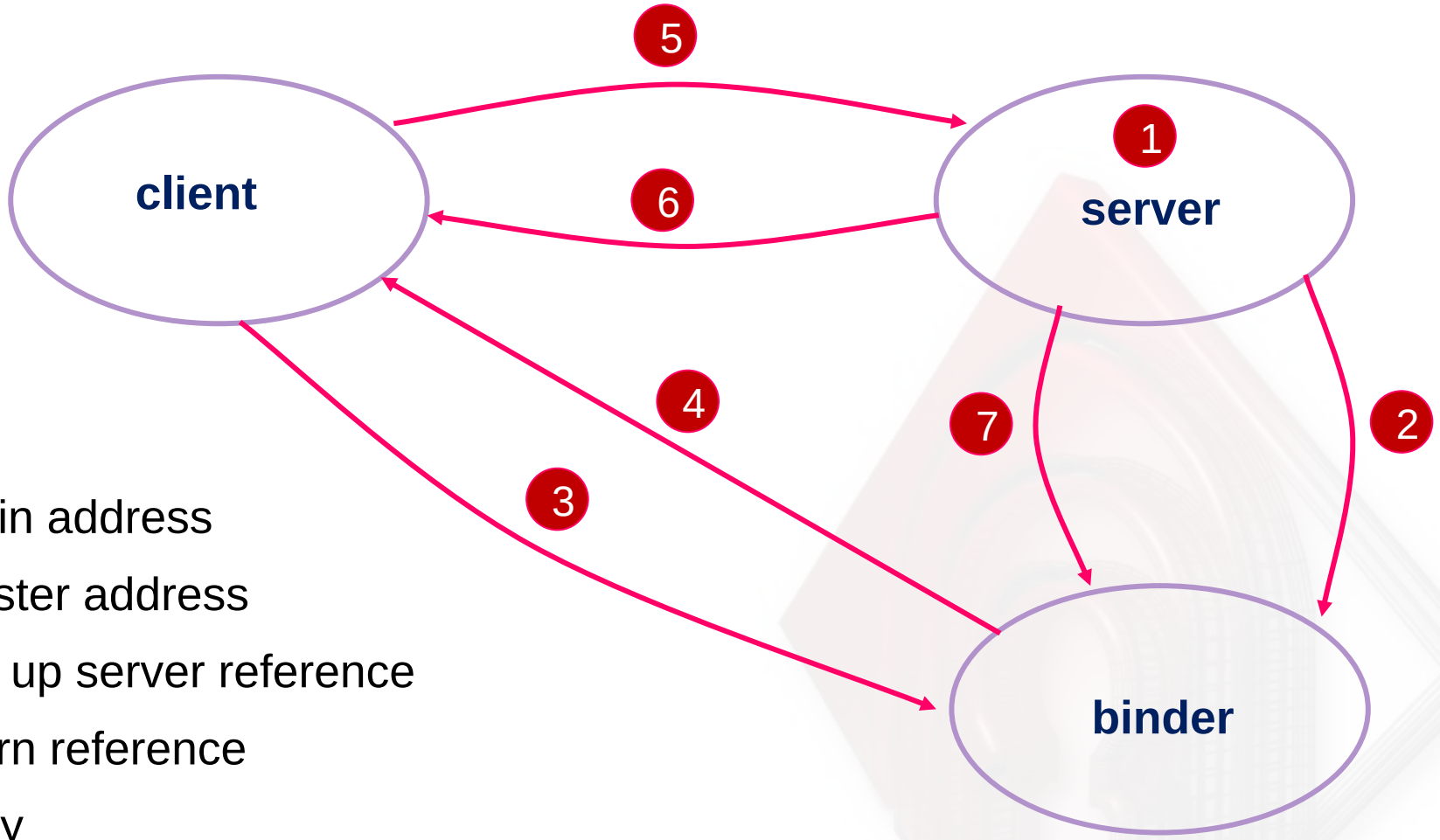
Binding

The mechanism to obtain the server reference.

Dynamic binding

- Servers register their references (addresses) in a name server (binder)
- The *binder* provides operations to:
  - Register/Remove a service
  - Find a reference from a service identifier
- Alternatives to find the *binder:*
  - Fixed address
  - OS provides address
  - By means multicast/broadcast discovery

Binding



1. Obtain address
2. Register address
3. Look up server reference
4. Return reference
5. Query
6. Reply
7. Remove reference (end of service)

Binding types

Temporary

- The binding is resolved per each remote call
- Better failure tolerance
- Allows to migrate server (reference changes)

Persistent

- References are reused for subsequent calls
- Better with frequent calls

Hybrid models

- Cache with aging/timeout

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

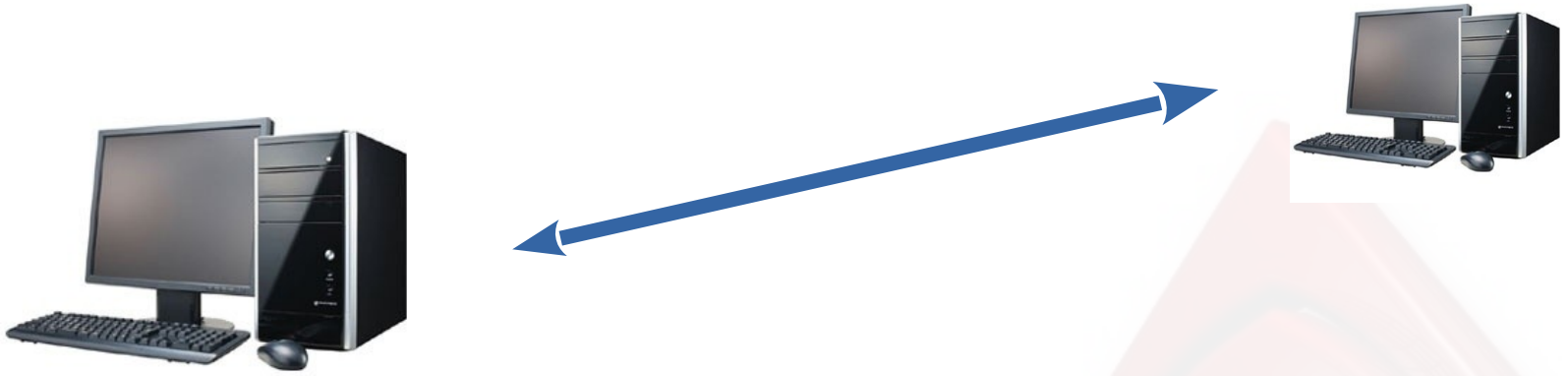
rpcbind (aka portmapper)

RFC 1833

- *Portmapper* is the binder used by ONC RPC
- Listen at port 111 (TCP/UDP)
- For each local server, it stores:
  - Program number
  - Version number
  - TCP/UDP port
- Protocol:
  - Any RPC server register itself on portmapper
  - Clients invoke portmapper giving program and version numbers
  - Portmapper reply with TCP/UDP port

rpcbind

DIY



```
$ rpcinfo -p localhost
   program vers proto   port  service
    100000    4   tcp    111  portmapper
    100000    3   tcp    111  portmapper
    100000    2   tcp    111  portmapper
    100000    4   udp    111  portmapper
    100000    3   udp    111  portmapper
    100000    2   udp    111  portmapper
 600000000    1   udp  36176
 600000000    1   tcp  43907
```

Registration and resolution

- **Registration**: servers tell the binder where they are at start-up
  - ONC RPC: program, version, protocol and port (`rpcinfo -p`)
  - Java RMI: the programmer calls `rebind`; the Registry stores the whole stub
  - Ice: the object adapter registers itself; the runtime picks the port
- **Resolution**: explicit in Java RMI (`lookup`) and ONC RPC (`rpcbind` query), done once
  - Ice: no call, the runtime asks the locator before connecting and caches the endpoints
- **Re-resolution** (Ice): the cache expires on failure, not by time
  - Connection fails: ask again and retry; a server can move with no exception
  - Only connection failures: a sent non-idempotent request keeps at-most-once
  - The Java RMI stub and the ONC RPC binding are never renewed

Synchronous vs. Asynchronous programming in RMI

Synchronous (blocking) is the default model for most RPC/RMI middlewares. Poor scalability when invocation amount and duration increase.

Asynchronous invocation decouples invocation and result collection. Client is not blocked at the time of invocation (don't be confused with *maybe* semantics). The client has three options:

PSEUDOCODE

Wait for request completion (pull)

```
async_result = obj.method()
[...]
result = async_result.get()
```

Provide a **callback** (push)

```
def listener(result):
    [...]

obj.method(callback=listener)
[...]
```

Poll for completion (polling)

```
async_result = obj.method()
while not async_result.ready():
    [...]

result = async_result.get()
```

async/await (push)

```
result = await obj.method()
[...]
```

Synchronous vs. Asynchronous programming in RMI

Asynchronous dispatching decouples incoming request and request dispatching.

- Server do not need a thread per each concurrent request.

ZeroC Ice provides both mechanisms:

- **AMI**: Asynchronous Method Invocation
  - https://github.com/UCLM-ESI/hello.ice/tree/master/py-ami
- **AMD**: Asynchronous Method Dispatching
  - https://github.com/UCLM-ESI/hello.ice/tree/master/py-amd

Asynchronous invocation and dispatching

- In a synchronous call the client thread is blocked from request to reply
- Two independent mechanisms, one at each end
  - Invocation: the client does not block. Dispatching: the server does not block
  - Neither end can tell whether the other uses its own: on the wire it is the same RR
- **AMI in Ice**: the proxy gets `factorialAsync`, which returns a *future*
  - Collect it later with `result`, with a callback, `await` or by polling
  - The exception travels in the future; the callback runs in a runtime thread
  - Semantics unchanged: still *at-most-once*; only when the client finds out changes

Asynchronous dispatching (AMD)

- The runtime serves requests with a limited thread pool
  - By default the thread is not free until the servant returns
  - A few slow requests can exhaust the pool, even with an idle node
- AMD decouples method return from sending the reply
  - The servant returns without the result; the runtime replies when it is available
- In Ice it shows in the interface: `["amd"] long factorial(int value)`
  - The servant returns a future; **`set_result`** or **`set_exception`** completes it
- The client cannot tell the difference
  - Pays off when the operation waits (database, remote service), not for pure computation

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Case study
Cap'n Proto

- Serialization format and RMI middleware in one
- Three decisions:
  - no serialization step,
  - *capabilities*, and
  - *promise pipelining*
- **No serialization step**: the message is built in a buffer with the wire layout
  - The receiver reads it as is; generated types are accessors over the buffer
  - Read one field without walking the rest; cost: bigger messages
- Interface: a 64-bit file id and an ordinal `@0` per method
  - Like the field numbers of Protocol Buffers: adding methods does not break compatibility

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Cap'n Proto
Capabilities

- A reference is a **capability**: it designates an object *and* grants permission to invoke it
  - Java RMI or Ice: a reference is data, knowing it is enough
  - Cap'n Proto: it cannot be forged, only an identifier valid in that connection is sent
- Access control lists: least privilege is enforced from outside
  - *Confused deputy*: a compiler with its own statistics file is told to write its output there
- With capabilities the client hands over the reference, not a name
  - Policy becomes interface design: hand over an object that only reads
  - As strong as the distribution: handing out too many capabilities defeats it
- No name service: objects are reached from a bootstrap object offered on connecting
  - Capabilities live in the connection: if it drops, start again from the bootstrap

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Cap'n Proto
Promise pipelining

- Every invocation returns a **promise** at once; it does not try to look like a local call
- Common pattern: invoke, get an object, invoke on it
  - `directory = root.open("bar")`
  - `file = directory.open("foo")`
  - `data = file.read()`
- Elsewhere each call waits for the previous reply: three round trips
- Cap'n Proto sends the three together, each referring to the previous result
  - One round trip, if all the objects are hosted by the same server
- **Levels**: 1 references and pipelining, 2 persistent capabilities,
  - 3 three-party interactions, 4 identity check between capabilities

Cap'n Proto Example

DIY

Description: A Cap'n Proto hello world example

- **`[ssdd.examples:capn-rpc-hello]`**

See and play with:
- **`server.py`**
- **`client.py`**

Also available in:
- **C++**

Interface:

`upper.capnp`

```
@0x8b96e1f756a41043;

interface TextProcessor {
  upper @0 (message :Text) -> (result :Text);
}
```

Run server:

```
py$ ./server.py 2000
- Server listening on port 2000
Client sent: hello, world!
```

Run client:

```
py$ ./client.py localhost 2000
Server reply: HELLO, WORLD!
```

Performance comparison

Mean response time per invocation (µs)

	Local call	Sockets	Middleware
Same process	**0.0075**	–	–
Same host	–	**15**	**45**
Remote host	–	**35 400**	**35 800**

Cost of a remote invocation

- `factorial(20)` through each mechanism: in-process, loopback and another node
  - Each middleware compared with raw sockets of its own language
- Crossing a process boundary costs orders of magnitude more than a local call
  - It happens before choosing a middleware
- Same node: ONC RPC the cheapest, gRPC the most expensive (a full HTTP/2 request)
  - The language matters: the same Ice in Python and in C++
- Across the network all converge to the round-trip time
  - What matters is *how many* calls, not how much each costs
- The first call costs much more: connection, code loading, compilation

Case study
REST (Representational State Transfer)

REST is an architectural style. An API that fulfills the principles of this style is a "RESTful API".

1. Uniform interface
2. Client-server
3. Stateless
4. Cacheable
5. Layered system

REST transfers **representations** of **resource states**

- Each resource is identified by a unique URI.
- State change are driven by **resource methods** (typically HTTP).

REST
Resource Methods

They mimic a CRUD interface:

- GET: retrieve resource representation
  - Don't modify state and it's idempotent
- POST: create a resource from representation
- PUT: update/replace existing resource
  - Idempotent
- DELETE: remove resource
  - Idempotent
- PATCH: partial update of resource

REST is not RPC

- It does not try to imitate local calls.
- There is no interface specification language, nor compiler.
- The same interface is used for any resource.
- Semantics are predefined.
- It is data-oriented, but does not exclude it as a remote invocation mechanism.

REST

DIY

Description: A simple REST example

- **[ssdd.examples:flask.rest]**

See and play with:
- **server.py**
- **client.py**

Run server:

```
flask.rest$ ./server.py
127.0.0.1 - - [19/Oct/2021 18:02:05] "GET /device1 HTTP/1.1" 404 -
127.0.0.1 - - [19/Oct/2021 18:02:05] "PUT /device1 HTTP/1.1" 200 -
127.0.0.1 - - [19/Oct/2021 18:02:05] "GET /device1 HTTP/1.1" 200 -
```

Run client:

```
flask.rest$ ./client.py
{'device1': 'enabled'}
```

More remote invocations...

- Web services: SOAP
- XML-RPC
- XMLHttpRequest
- JSON RPC

```
Request: {"jsonrpc": "1.2", "id": 1, "method": "factorial", "params": [6]}
Reply:   {"jsonrpc": "1.2", "id": 1, "result": 720}
```

What have you learned?

- **Remote Invocation** is the base of many distributed systems and apps.
- Importance of exchange protocols. R/RR/RRA
- Invocation semantics: maybe, at-least-one, at-most-once
- Provide **transparency** to use remote resources.
  - Transparent un/marshalling
  - Transparent transport
  - Access and Location transparency
- IDLs provide implementation language transparency.

References

G. Coulouris, *Distributed Systems: Concepts and Design*, Addison Wesley 2011

- Section 4.1 – Intro
- Section 4.2 – Internet technology review
  (may skip Java study case)
- Section 5.2 – Request-reply protocols
- Section 5.3.1 – Design issues for RPC
- Section 5.4 – Remote Method Invocation

References

- A.D.Birrell, B.J. Nelson, “Implementing Remote Procedure Calls”, Xerox PARC
- Example:
  - http://docs.oracle.com/cd/E19683-01/816-1435/rpcgenpguide-21470/index.html
- RPC:
  - https://www.dre.vanderbilt.edu/~schmidt/PDF/rpc4.pdf
- XDR data types and formats
  - http://pubs.opengroup.org/onlinepubs/9629799/chap3.htm